

# IF440: Artificial Intelligence

## Topic 13: Reinforcement Learning

Moeljono Widjaja, Ph.D.  
Yaya Suryana, Ph.D.

**Universitas Multimedia Nusantara**  
Jl. Scientia Boulevard, Gading Serpong  
Tangerang, Banten-15811 Indonesia

Odd Semester 2025-2026

MULTIMEDIA  
NUSANTARA

## Course Sub Learning Outcomes (Sub-CLO):



SubCLO-0221: Students are able to apply knowledge-based reasoning, probabilistic reasoning and soft computing (C3).



# Review

- 1 Introduction
- 2 Basic Components
- 3 Algorithm Flow
- 4 Detailed Examples
- 5 Practical Example
- 6 Strengths and Limitations
- 7 Applications



# Pre-class Assignment

- deeplizard: Reinforcement Learning Series Intro - Syllabus Overview  
[https://youtu.be/nyjbcRQ-uQ8?si=D4tZM5Hmn\\_pmBGXR](https://youtu.be/nyjbcRQ-uQ8?si=D4tZM5Hmn_pmBGXR)
- deeplizard: Markov Decision Processes (MDPs) - Structuring a Reinforcement Learning Problem  
<https://youtu.be/my207WNoeyA?si=3eU-5TsJiMX0KSBF>
- deeplizard: Expected Return - What Drives a Reinforcement Learning Agent in an MDP <https://youtu.be/a-SnJtmBtyA?si=HQeRGpPsRe6YnXct>
- deeplizard: Policies and Value Functions - Good Actions for a Reinforcement Learning Agent [https://youtu.be/eMxOGwbdqKY?si=3XO\\_qz39crVZLqYV](https://youtu.be/eMxOGwbdqKY?si=3XO_qz39crVZLqYV)
- deeplizard: What do Reinforcement Learning Algorithms Learn - Optimal Policies  
<https://youtu.be/rP4oEpQbDm4?si=64zk9IsQCUS84CRL>
- deeplizard: Q-Learning Explained - A Reinforcement Learning Technique  
<https://youtu.be/qhRNvCVVJaA?si=dRxe3VpB9NA0bLh4>

# Outline

- 1 Introduction
- 2 Markov Decision Process
- 3 Q Learning
- 4 Summary





# Introduction

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Why Reinforcement Learning?

- Learns by interaction with an environment (trial-and-error).
- Useful for sequential decision-making where actions affect future states and rewards.
- Four main sub-elements of a reinforcement learning system: a policy, a reward signal, a value function, and, optionally, a model of the environment.
- Applications: robotics, game playing, resource allocation, recommendation systems.

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Examples of Reinforcement Learning

- How should a robot behave so as to optimize its “performance”? (Robotics)



- How to automate the motion of a helicopter? (Control Theory)



- How to make a good chess-playing program? (Artificial Intelligence)





# Supervised Learning vs Reinforcement Learning

## ■ Supervised learning

$$f : X \rightarrow Y$$

- X: inputs
- Y: outputs
- The predicted outputs can be evaluated immediately by the teacher
- $f$  evaluated with loss function

## ■ Reinforcement learning for learning a policy

$$\pi : S \mapsto A$$

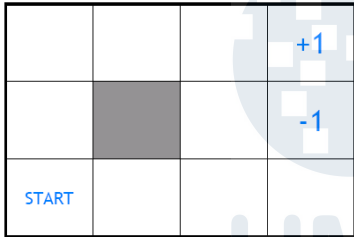
- S: states
- A: actions
- Take action A to affect state S
- The predicted action A cannot be evaluated directly but can be given positive/negative rewards
- Policy evaluated with value functions

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

## Elements of RL

- **policy**  $\pi : S \mapsto A$ 
  - A map from **state space** to **action space**.
  - May be stochastic.
- **reward function**  $R(S)$ 
  - It maps each state (or, state-action pair) to a real number, called **reward**.
- **value function**
  - Value of a state (or, state-action pair) is the **total expected reward**, starting from that state (or, state-action pair).

## Robot in a room



actions: UP, DOWN, LEFT, RIGHT

UP

80% move UP  
10% move LEFT  
10% move RIGHT



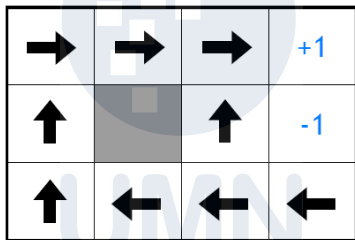
- reward +1 at [4,3], -1 at [4,2]
- reward -0.04 for each step
- what's the strategy to achieve max reward?
- what if the actions were NOT deterministic?

## Policy

|   |   |   |    |
|---|---|---|----|
| → | → | → | +1 |
| ↑ |   | ↑ | -1 |
| ↑ | ← | ← | ←  |

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

**Reward for each step: -0.04**



|   |   |   |    |
|---|---|---|----|
| → | → | → | +1 |
| ↑ |   | ↑ | -1 |
| ↑ | ← | ← | ←  |

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

## The Precise Goal

- To find a **policy**  $\pi : S \mapsto A$  that maximizes the **Value function**.
- There are different approaches to achieve this goal in various situations.
- **Value iteration** and **Policy iteration** are two more classic approaches to this problem. But essentially both are **dynamic programming**.
- **Q-learning** is a more recent approaches to this problem. Essentially it is a **temporal-difference method**.



# Markov Decision Process

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Markov Decision Process (MDP) – Intuition

- 
- An MDP formalizes sequential decision problems.
  - Components: states, actions, transition dynamics, rewards, and discounting.
  - The Markov property: the future is independent of the past given the present state.

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA



# MDP – Formal Definition

An MDP is a tuple  $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$  where:

- $\mathcal{S}$ : set of states.
- $\mathcal{A}$ : set of actions (can be state-dependent  $\mathcal{A}(s)$ ).
- $P(s' | s, a)$ : transition probability of next state  $s'$  given  $(s, a)$ .
- $R(s, a)$  or  $R(s, a, s')$ : immediate reward function.
- $\gamma \in [0, 1]$ : discount factor for future rewards.

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Objective: Return and Optimality

- A trajectory:  $s_0, a_0, r_1, s_1, a_1, r_2, \dots$
- Return (discounted):  $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ .
- Goal: find a policy  $\pi(a | s)$  that maximizes expected return  $\mathbb{E}[G_0]$ .
- Optimal value functions:

$$v^*(s) = \max_{\pi} \mathbb{E}_{\pi}[G_t | S_t = s],$$
$$q^*(s, a) = \max_{\pi} \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a].$$

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# SIMPLE MDP: GRID WORLD

actions = {

1. right →

2. left ←

3. up ↑

4. down ↓

}

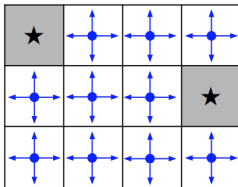
states

|   |  |  |   |
|---|--|--|---|
| ★ |  |  |   |
|   |  |  | ★ |
|   |  |  |   |

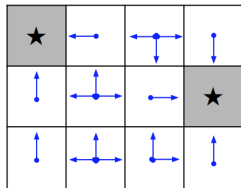
Set a negative “reward”  
for each transition  
(e.g.  $r = -1$ )

- **Objective:** reach one of terminal states (greyed out) in least number of actions

# SIMPLE MDP: GRID WORLD



Random Policy



Optimal Policy

- **Objective:** reach one of terminal states (greyed out) in least number of actions

# SIMPLE MDP: GRID WORLD

- We want to find optimal policy  $\pi^*$  that maximizes the sum of rewards.
- How do we handle the randomness (initial state, transition probability...)?

**Maximize the expected sum of rewards!**



# DEFINITIONS: VALUE FUNCTION AND Q-VALUE FUNCTION

- Following a policy produces sample trajectories (or paths)  $s_0, a_0, r_0, s_1, a_1, r_1, \dots$
- How good is a state? The **value function** at state  $s$ , is the expected cumulative reward from following the policy from state  $s$ :

$$V^\pi(s) = \mathbb{E} \left[ \sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, \pi \right]$$

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# DEFINITIONS: VALUE FUNCTION AND Q-VALUE FUNCTION

- How good is a state-action pair? The **Q-value function** at state  $s$  and action  $a$ , is the expected cumulative reward from taking action  $a$  in state  $s$  and then following the policy:

$$Q^{\pi}(s, a) = \mathbb{E} \left[ \sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$

# BELLMAN EQUATION

- The optimal Q-value function  $Q^*$  is the **maximum** expected cumulative reward achievable from a given (state, action) pair:

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[ \sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$

- $Q^*$  satisfies the following Bellman equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[ r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$



# BELLMAN EQUATION

- **Intuition:** if the optimal state-action values for the next time-step  $Q^*(s',a')$  are known, then the optimal strategy is to take the action that maximizes the expected value of
$$r + \gamma Q^*(s', a')$$

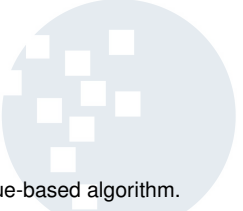
- The optimal policy  $\pi^*$  corresponds to taking the best action in any state as specified by  $Q^*$



## Q Learning

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Q-Learning — Overview

- 
- Off-policy, model-free value-based algorithm.
  - Learns an estimate  $Q(s, a)$  of the optimal action-value function  $q^*(s, a)$ .
  - Update uses Bellman optimality backup.

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Q-Learning Update Rule

For each transition  $(s, a, r, s')$ :

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)).$$

- $\alpha$  is the learning rate.
- Uses max over next-state actions (hence off-policy).
- With tabular representation and suitable exploration, Q-Learning converges to  $q^*$ .

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA

# Q-Learning — Pseudocode

---

## Algorithm Q-Learning

---

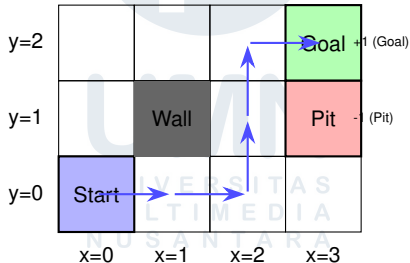
```
1: Initialize  $Q(s, a)$  arbitrarily (e.g., zeros)
2: for each episode do
3:   Initialize state  $s$ 
4:   for each step of episode do
5:     Choose action  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
6:     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
7:      $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$ 
8:      $s \leftarrow s'$ 
9:     if  $s'$  is terminal then
10:      break
11:    end if
12:  end for
13: end for
```

---

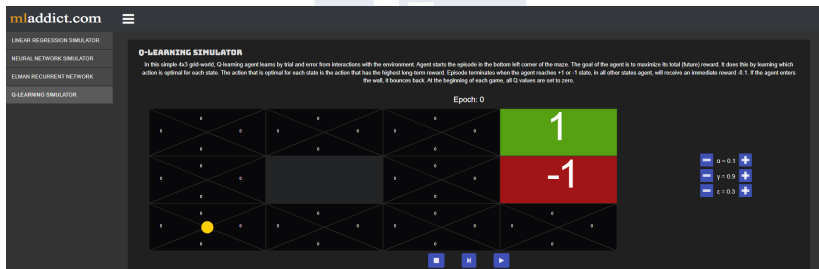
# Simple Example: Gridworld

- States: grid cells; Actions: up/down/left/right.
- Rewards: +1 for reaching goal, -1 for stepping on a trap, -0.1 otherwise.
- Use Q-Learning to learn optimal navigation policy.

**3×4 Gridworld**



# Q-Learning Simulator



<https://www.mladdict.com/q-learning-simulator>

UNIVERSITAS  
MULTIMEDIA  
NUSANTARA



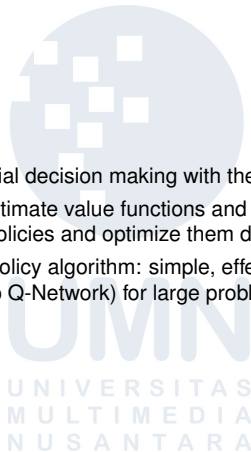
## Summary

UMN  
UNIVERSITAS  
MULTIMEDIA  
NUSANTARA



# Summary

- MDPs formalize sequential decision making with the Markov property.
- Value-based methods estimate value functions and derive policies; policy-based methods parameterize policies and optimize them directly.
- Q-Learning is a key off-policy algorithm: simple, effective in tabular settings, and extensible to DQN (Deep Q-Network) for large problems.

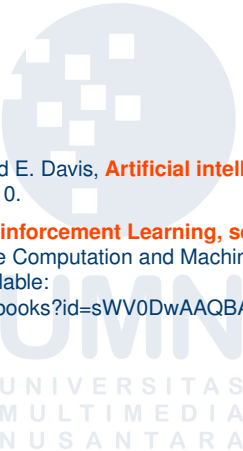


# Simulator

- **Temporal Difference Learning Gridworld Demo** [https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld\\_td.html](https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_td.html)
- **GridWorld: Dynamic Programming Demo** [https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld\\_dp.html](https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html)
- **Q-Learning Simulator**  
<https://www.mladdict.com/q-learning-simulator>



# References

- 
- [1] S. J. Russell, P. Norvig, and E. Davis, **Artificial intelligence: a modern approach**, 3rd ed. Prentice Hall, 2010.
- [2] R. Sutton and A. Barto, **Reinforcement Learning, second edition: An Introduction**, ser. Adaptive Computation and Machine Learning series. MIT Press, 2018. [Online]. Available: <https://books.google.co.id/books?id=sWV0DwAAQBAJ>

# Next Week

- Deep Learning
- Training Rules
- Activation Functions
- Normalization
- Optimizers
- Neural Network Types



# Vision and Mission

## Vision

Becoming a leading Informatics Undergraduate Study Program that produces graduates with international insight who are competent in the field of computer science, have an entrepreneurial spirit, and have noble character.



## Mission

- 1 Organising learning with the best technology and curriculum and supported by professional teaching staff.
- 2 Carry out research activities in the field of informatics to advance information science and technology.
- 3 Carry out community service activities based on information technology and science in the context of practising informatics science and technology society.

